

Django Application Deployment to AWS EC2 with PostgreSQL, Nginx, and Gunicorn

Table of Contents

- Part 1: Prepare Your Django Application
- Part 2: Create AWS Server
- Part 3: Connect & Install Software
- Part 4: Setup PostgreSQL Database
- Part 5: Clone Your Project
- Part 6: Setup Gunicorn
- Part 7: Configure Nginx
- Part 8: Test Your Application
- Part 9: Add Domain and SSL
- Part 10: Deploy Updates
- Part 11: Monitoring and Maintenance
- Troubleshooting Guide
- Deployment Checklist

Before You Start

Before beginning this deployment process, ensure you have the following prerequisites in place:

- Django application ready with PostgreSQL support
- GitHub account with your project repository
- AWS account with billing information configured
- Terminal or Command Prompt access on your local machine
- Basic familiarity with command-line operations
- SSH client (built-in on Linux/Mac, or PuTTY for Windows)

Part 1: Prepare Your Django Application

1.1 Update requirements.txt

Add the following packages to your requirements.txt file. These packages are essential for production deployment on AWS:

- Django>=4.2.0
- psycopg2-binary==2.9.9
- gunicorn==21.2.0
- whitenoise==6.5.0
- python-decouple==3.8

1.2 Update Django settings.py

Modify your Django project's settings.py file to support production deployment. The following changes are critical:

- Import configuration modules at the top of the file
- Use environment variables for sensitive data (SECRET_KEY, passwords)
- Set DEBUG = False for production environments
- Configure ALLOWED_HOSTS with your server IP and domain
- Update DATABASES configuration to use PostgreSQL
- Add Whitenoise to INSTALLED_APPS and MIDDLEWARE
- Configure static files paths for production serving
- Set security-related settings for production use

1.3 Create .env.example File

Create a .env.example file in your project root to document required environment variables. This file should NOT contain actual credentials—it serves as a template. Add this file to version control but keep the actual .env file in .gitignore.

1.4 Verify wsgi.py Configuration

Ensure your Django project contains a wsgi.py file (typically in your project folder or apps/config/ directory). This file should be left unchanged—it serves as the entry point for Gunicorn to run your Django application.

1.5 Update .gitignore

Ensure sensitive files are not committed to version control:

- .env (environment variables with secrets)
- *.pyc (compiled Python files)
- __pycache__/ (Python cache)
- *.sqlite3 (local database)
- staticfiles/ (collected static files)
- media/ (uploaded files)
- venv/ (virtual environment)

1.6 Push to GitHub

Commit all configuration changes and push to your GitHub repository. This prepares your code for deployment to AWS.

Part 2: Create AWS Server

2.1 Launch EC2 Instance

Follow these steps to create an EC2 instance on AWS:

- Navigate to AWS Console and select EC2 service
- Click 'Launch Instances' button
- Select Ubuntu 22.04 LTS as the operating system (free tier eligible)
- Choose t2.micro as instance type (free tier)
- Configure storage to 30 GB (free tier allocation)
- Create security group with the following rules:

Port	Protocol	Access Level	Purpose
22	SSH	Your IP only	Server terminal access
80	HTTP	Anywhere	Web traffic
443	HTTPS	Anywhere	Secure web traffic
8000	TCP	Anywhere	Testing (optional)

- Create a new SSH key pair (download and save securely)
- Click 'Launch Instance' to start the server
- Wait for instance to reach 'running' status
- Copy the Public IPv4 address for later use

Part 3: Connect to Server & Install Software

3.1 Connect via SSH

Use the SSH key you downloaded to connect to your AWS server from your local machine. Open your terminal or command prompt and execute the following command:

```
ssh -i your-key.pem ubuntu@YOUR_PUBLIC_IP
```

3.2 Update System

Once connected to your server, update all system packages to the latest versions:

```
sudo apt update  
sudo apt upgrade -y
```

3.3 Install Required Software

Install all necessary packages for Django deployment:

```
sudo apt install python3.11 python3-pip python3-venv -y  
sudo apt install postgresql postgresql-contrib -y  
sudo apt install nginx -y  
sudo apt install git -y  
sudo apt install build-essential libpq-dev -y
```

3.4 Start PostgreSQL

Enable and start the PostgreSQL database service:

```
sudo systemctl start postgresql  
sudo systemctl enable postgresql
```

Part 4: Setup PostgreSQL Database

4.1 Create Database User and Database

Connect to PostgreSQL and create a database user and database for your Django application. Execute the following command to open PostgreSQL console:

```
sudo -u postgres psql
```

Inside PostgreSQL, execute these SQL commands (one at a time):

```
CREATE USER django_user WITH PASSWORD 'your_strong_password_here';  
CREATE DATABASE django_db OWNER django_user;  
ALTER ROLE django_user SET client_encoding TO 'utf8';  
ALTER ROLE django_user SET default_transaction_isolation TO 'read committed';  
ALTER ROLE django_user SET timezone TO 'UTC';  
\\q (to exit PostgreSQL)
```

Note: Remember the password you set for `django_user`—you will need it in the `.env` file later.

Part 5: Clone Your Project & Setup

5.1 Create Project Directory

```
cd /home/ubuntu  
mkdir myproject  
cd myproject
```

5.2 Clone from GitHub

Clone your Django project repository from GitHub:

```
git clone https://github.com/your-username/your-repo.git .
```

5.3 Create Virtual Environment

Create and activate a Python virtual environment for isolated package management:

```
python3.11 -m venv venv  
source venv/bin/activate  
pip install --upgrade pip  
pip install -r requirements.txt
```

5.4 Create .env File

Create a .env file on your server with the actual database credentials and configuration:

```
nano .env
```

Add the following content (with your actual values):

```
DEBUG=False  
SECRET_KEY=your-very-long-random-secret-key  
DB_NAME=django_db  
DB_USER=django_user  
DB_PASSWORD=your_password  
DB_HOST=localhost  
DB_PORT=5432  
ALLOWED_HOSTS=YOUR_PUBLIC_IP,localhost  
CSRF_TRUSTED_ORIGINS=https://YOUR_PUBLIC_IP
```

Save: Ctrl+X, then Y, then Enter

5.5 Run Django Setup

Initialize your Django application by running migrations, collecting static files, and creating a superuser:

```
python manage.py migrate  
python manage.py collectstatic --noinput  
python manage.py createsuperuser
```

5.6 Test Django

Test if Django is running correctly by starting the development server:

```
python manage.py runserver 0.0.0.0:8000
```

Visit http://YOUR_PUBLIC_IP:8000 in your browser. If you see your Django application, press Ctrl+C to stop the server.

Part 6: Setup Gunicorn (Application Server)

6.1 Create Gunicorn Service File

Create a systemd service file that will manage the Gunicorn process and ensure it starts automatically:

```
sudo nano /etc/systemd/system/gunicorn.service
```

Paste the following configuration:

```
[Unit]

Description=gunicorn daemon for Django

After=network.target


[Service]

User=ubuntu

Group=www-data

WorkingDirectory=/home/ubuntu/your-project

Environment="PATH=/home/ubuntu/your-project/venv/bin"

ExecStart=/home/ubuntu/your-project/venv/bin/gunicorn \

--workers 3 \

--bind unix:/run/gunicorn.sock \

your_project_name.wsgi:application


Restart=always

RestartSec=10


[Install]

WantedBy=multi-user.target
```

Important: Replace 'your_project_name.wsgi:application' with your actual WSGI module path. Save: Ctrl+X, Y, Enter

6.2 Enable and Start Gunicorn

```
sudo systemctl daemon-reload

sudo systemctl enable gunicorn

sudo systemctl start gunicorn

sudo systemctl status gunicorn
```

Part 7: Configure Nginx (Web Server)

7.1 Create Nginx Configuration

Create an Nginx configuration file that acts as a reverse proxy:

```
sudo nano /etc/nginx/sites-available/django
```

Paste the following configuration:

```
upstream gunicorn {  
    server unix:/run/gunicorn.sock;  
}  
  
server {  
    listen 80 default_server;  
    listen [::]:80 default_server;  
  
    server_name YOUR_IP;  
  
    client_max_body_size 20M;  
  
    location /static/ {  
        alias /home/ubuntu/your-project/staticfiles/;  
        expires 30d;  
    }  
  
    location / {  
        proxy_pass http://gunicorn;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Save: Ctrl+X, Y, Enter

7.2 Enable Nginx Configuration

```
sudo ln -s /etc/nginx/sites-available/django /etc/nginx/sites-enabled/
```

```
sudo rm /etc/nginx/sites-enabled/default
```

```
sudo nginx -t
```

```
sudo systemctl restart nginx
```

```
sudo systemctl enable nginx
```

Part 8: Test Your Application

8.1 Verify All Services

Check that all required services are running:

```
sudo systemctl status gunicorn  
sudo systemctl status nginx  
sudo systemctl status postgresql
```

All services should show 'active (running)'.

8.2 Test in Web Browser

Open your web browser and visit your server:

```
http://YOUR_PUBLIC_IP
```

You should see your Django application.

```
Admin panel: http://YOUR_PUBLIC_IP/admin
```

8.3 Check Logs if Issues Occur

```
Django/Gunicorn: sudo journalctl -u gunicorn -n 50  
Nginx: sudo tail -f /var/log/nginx/error.log  
Access logs: sudo tail -f /var/log/nginx/access.log
```

Part 9: Add Domain and SSL (Optional)

9.1 Point Domain to AWS

If you have a domain, configure DNS to point to your AWS server's public IP address. Update your server's `.env` file:

```
nano .env
```

Change `ALLOWED_HOSTS` to include your domain:

```
ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com
```

```
Restart Unicorn: sudo systemctl restart gunicorn
```

9.2 Get SSL Certificate

Install Certbot and obtain a free SSL certificate from Let's Encrypt:

```
sudo apt install certbot python3-certbot-nginx -y
```

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Your site is now accessible via HTTPS.

Part 10: Deploy Code Updates

10.1 Update Workflow

When you make changes to your Django application, follow this workflow:

- On your local machine: `git push origin main`
- SSH to server: `ssh -i django-key.pem ubuntu@YOUR_IP`
- Navigate to project: `cd /home/ubuntu/your-project`
- Activate venv: `source venv/bin/activate`
- Pull latest code: `git pull origin main`
- Install new packages: `pip install -r requirements.txt`
- Run migrations: `python manage.py migrate`
- Collect static files: `python manage.py collectstatic --noinput`
- Restart services: `sudo systemctl restart gunicorn nginx`

Your changes are now live in production.

Part 11: Monitoring and Maintenance

11.1 Check Server Health

Disk space: `df -h`

Memory usage: `free -h`

Services status: `sudo systemctl status gunicorn nginx postgresql`

11.2 View Logs

Monitor application logs for errors and issues:

Django logs: `sudo journalctl -u gunicorn -n 30`

Real-time logs: `sudo journalctl -u gunicorn -f`

Nginx errors: `sudo tail -f /var/log/nginx/error.log`

11.3 Database Backups

Regularly backup your PostgreSQL database:

Create backup: `pg_dump -U django_user django_db > /home/ubuntu/backup.sql`

Restore backup: `psql -U django_user django_db < /home/ubuntu/backup.sql`

Deployment Checklist

Before launching your application to production, verify the following:

- ✓ Django settings.py configured for production (DEBUG=False)
- ✓ All environment variables defined in .env file
- ✓ requirements.txt includes all necessary packages
- ✓ .gitignore prevents secrets from being committed
- ✓ AWS EC2 instance running with correct security groups
- ✓ PostgreSQL database and user created on server
- ✓ Django project cloned to server
- ✓ Virtual environment created and activated
- ✓ Python packages installed from requirements.txt
- ✓ Django migrations executed successfully
- ✓ Static files collected
- ✓ Superuser created for admin access
- ✓ Gunicorn service file created and enabled
- ✓ Gunicorn service running (sudo systemctl status gunicorn)
- ✓ Nginx configuration created and enabled
- ✓ Nginx service running (sudo systemctl status nginx)
- ✓ Application accessible at http://YOUR_IP
- ✓ Admin panel accessible at http://YOUR_IP/admin
- ✓ Static files loading correctly (CSS, images)
- ✓ Database queries functioning properly
- ✓ All error logs checked and cleaned
- ✓ Domain configured (if applicable)
- ✓ SSL certificate installed (if applicable)

Quick Reference Guide

Essential Commands

Connection:

```
ssh -i django-key.pem ubuntu@YOUR_PUBLIC_IP
```

Service Management:

```
sudo systemctl restart gunicorn
sudo systemctl restart nginx
sudo systemctl status gunicorn
sudo systemctl status nginx
sudo systemctl status postgresql
```

Logging and Monitoring:

```
sudo journalctl -u gunicorn -n 30
sudo journalctl -u gunicorn -f
sudo tail -f /var/log/nginx/error.log
df -h
free -h
```

Code Deployment:

```
git pull origin main
pip install -r requirements.txt
python manage.py migrate
python manage.py collectstatic --noinput
sudo systemctl restart gunicorn nginx
```

Database Operations:

```
pg_dump -U django_user django_db > backup.sql
psql -U django_user django_db < backup.sql
```

System Architecture

The following diagram illustrates the flow of traffic through your deployed Django application:

1. User Browser (Client makes HTTP request)
2. Nginx Web Server (Receives request on port 80/443)
3. Gunicorn Application Server (Routes to Django via Unix socket)
4. Django Application (Processes request, executes business logic)
5. PostgreSQL Database (Stores and retrieves data)
6. Response (Travels back through stack to browser)

Static files (CSS, JavaScript, images) are served directly by Nginx, bypassing Django for performance. Dynamic content is processed by Django and returned through Gunicorn.

Support Resources and Documentation

The following resources provide comprehensive documentation and support for the tools and services used in this deployment:

- Django Official Documentation: <https://docs.djangoproject.com/>
- AWS EC2 Documentation: <https://docs.aws.amazon.com/ec2/>
- PostgreSQL Documentation: <https://www.postgresql.org/docs/>
- Nginx Documentation: <https://nginx.org/en/docs/>
- Gunicorn Documentation: <https://docs.gunicorn.org/>
- Let's Encrypt SSL: <https://letsencrypt.org/>
- Python Virtual Environments: <https://docs.python.org/3/tutorial/venv.html>